

Tugas 1 – Analisis Arsitektur Web Service

Studi Kasus: Aplikasi Wisata (WisataKu)

Nama: Sulistiani Nasya Marzu (Nasya) | NIM: SI21240018

Program Studi Sistem Informasi – STMIK Lombok

1. Analisis Pemilihan Arsitektur Monolitik Modular

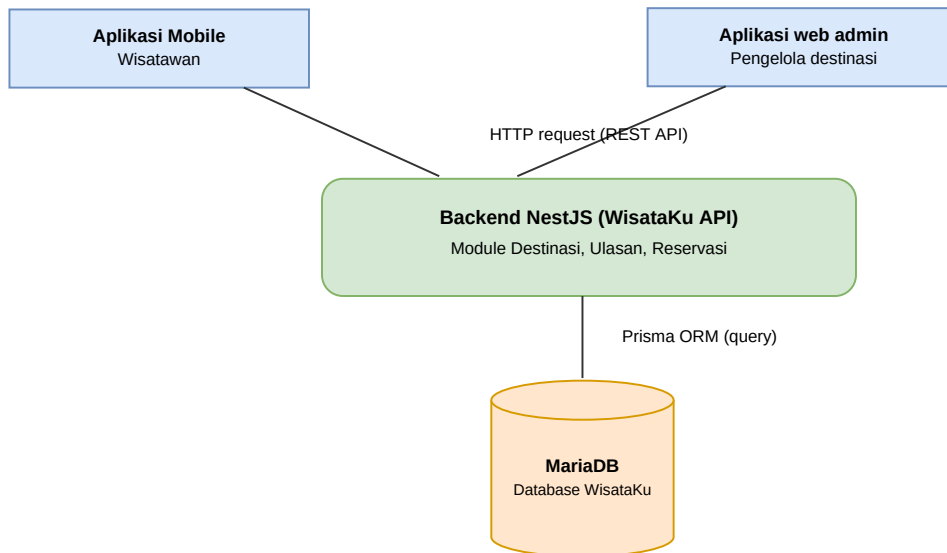
Mengapa arsitektur monolitik modular dipilih sebagai tahap awal pengembangan WisataKu, bukan langsung microservices?

- 1 **Skala tim masih kecil.** Di tahap awal, proyek WisataKu dikerjakan oleh individu/tim kecil. Arsitektur monolitik lebih efisien untuk tim kecil karena tidak perlu mengelola banyak service, banyak deployment, maupun komunikasi antar-layanan yang kompleks.
- 2 **Kecepatan pengembangan lebih tinggi.** Dengan satu basis kode (single codebase), fitur-fitur seperti modul Destinasi, Ulasan, dan Reservasi dapat dikembangkan dan diuji lebih cepat tanpa overhead pengaturan infrastruktur microservices (message broker, service discovery, dan sebagainya).
- 3 **Kompleksitas operasional lebih rendah.** Monolitik hanya membutuhkan satu proses deployment dan satu basis data, sehingga lebih mudah di-debug dibandingkan microservices yang membutuhkan monitoring terdistribusi, menghadapi network latency antar service, dan penanganan failure yang lebih rumit.
- 4 **Batas domain sudah jelas sejak awal (modular).** NestJS mendorong struktur berbasis Module, sehingga modul Destinasi, Ulasan, dan Reservasi sudah terpisah rapi meski berjalan dalam satu aplikasi. Hal ini membuat proses evolusi menuju microservices di kemudian hari menjadi lebih mudah, karena modul yang sudah terpisah tinggal “dilepas” menjadi service independen (sebagaimana dibahas pada Bab 6 modul).
- 5 **Sesuai dengan skala kebutuhan saat ini.** Microservices baru benar-benar dibutuhkan ketika aplikasi sudah berskala besar, traffic tinggi, dan tim pengembang sudah terpisah per domain. WisataKu di tahap awal belum membutuhkan skalabilitas per-layanan yang ditawarkan oleh microservices.

Kesimpulan: Monolitik modular dipilih sebagai strategi bertahap (evolutionary architecture) — mudah dikembangkan pada tahap sekarang, namun tetap siap dipecah menjadi microservices apabila aplikasi WisataKu sudah berkembang lebih besar di kemudian hari.

2. Diagram Arsitektur Akses Backend WisataKu

Diagram berikut menggambarkan bagaimana Aplikasi Mobile (wisatawan) dan Aplikasi Web Admin (pengelola destinasi) sama-sama mengakses satu Backend NestJS yang sama melalui HTTP request (REST API), dan bagaimana backend tersebut terhubung ke basis data MariaDB melalui Prisma ORM.



Gambar 1. Diagram arsitektur akses Backend WisataKu